Class Diagram

- Write a **code first** and convert the code into the class diagram.

1.Create a **Customer class:** will hold basic customer information such as the first name and last name.
2.Create a **Account class:**will hold information such as the account number, account balance and will extend the customer class to allow customer details to be used within the constructor. This class will also hold functions such as deposit and withdrawal.
3.Create a **Transaction class:** will hold a method called transfer to allow accounts to transfer money between them.
4.Create a **Test class:**will hold the objects of the above classes and perform the operation needed.

```java
// Customer.java
public class Customer {
    protected String firstName;
    protected String lastName;

    public Customer(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }

    public void displayCustomerInfo() {
           System.out.println("Customer Name: " + firstName + " " +
lastName);
    }
}
```

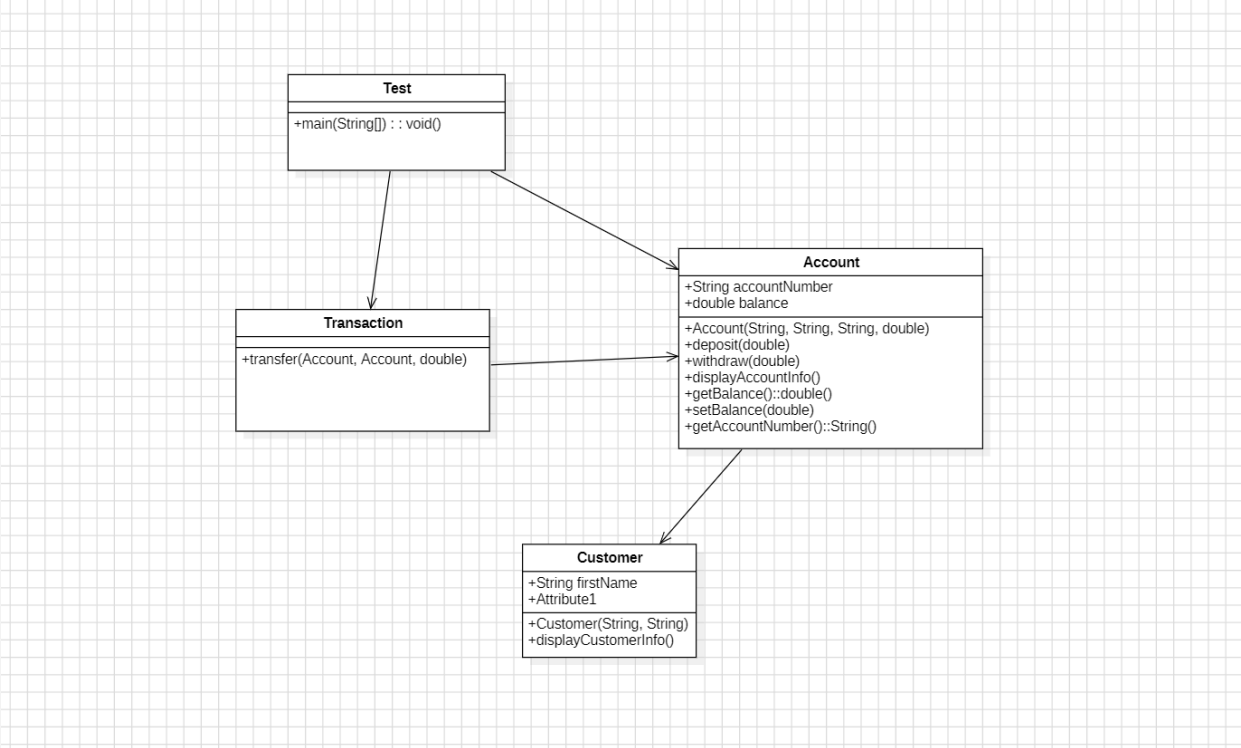
```
// Account.java
public class Account extends Customer {
    private String accountNumber;
    private double balance;

        public  Account(String  firstName,  String  lastName,  String
accountNumber, double balance) {
        super(firstName, lastName);
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: " + amount);
    }

    public void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient balance!");
        }
    }

    public void displayAccountInfo() {
        displayCustomerInfo();
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Balance: " + balance);
    }

    public double getBalance() {
        return balance;
    }
```

```
    public void setBalance(double amount) {
        this.balance = amount;
    }

    public String getAccountNumber() {
        return accountNumber;
    }
}
```

```
// Transaction.java
public class Transaction {
      public void transfer(Account fromAccount, Account toAccount,
double amount) {
        if (fromAccount.getBalance() >= amount) {
            fromAccount.withdraw(amount);
            toAccount.deposit(amount);
            System.out.println("Transferred " + amount + " from " +
                               fromAccount.getAccountNumber() + " to " +
toAccount.getAccountNumber());
        } else {
             System.out.println("Transfer failed due to insufficient
funds.");
        }
    }
}
```

```
// Test.java
public class Test {
    public static void main(String[] args) {
              Account acc1 = new Account("Yajju", "Chansi", "A001",
100000000.0);
         Account acc2 = new Account("Meri pyari Looza", "Yestaii ho hai",
"A002", 50000.0);

        acc1.displayAccountInfo();
```

```
        acc2.displayAccountInfo();

        Transaction transaction = new Transaction();

        transaction.transfer(acc1, acc2, 300.0);

        System.out.println("\nAfter Transfer:");
        acc1.displayAccountInfo();
        acc2.displayAccountInfo();
    }
}
```



UseCase Diagram

1. What are the primary actors described by the code above?

User (e.g., a bank customer using the system via the Test class)

2.What are the primary use cases (actions) described by the code above.

The actions that can be performed:

- **Create Account**
- **Deposit Money**
- **Withdraw Money**
- **Transfer Money**
- **View Account Info**

3.Assocations in the code above

User ↔ Create Account

User ↔ Deposit Money

User ↔ Withdraw Money

User ↔ Transfer Money

User ↔ View Account Info

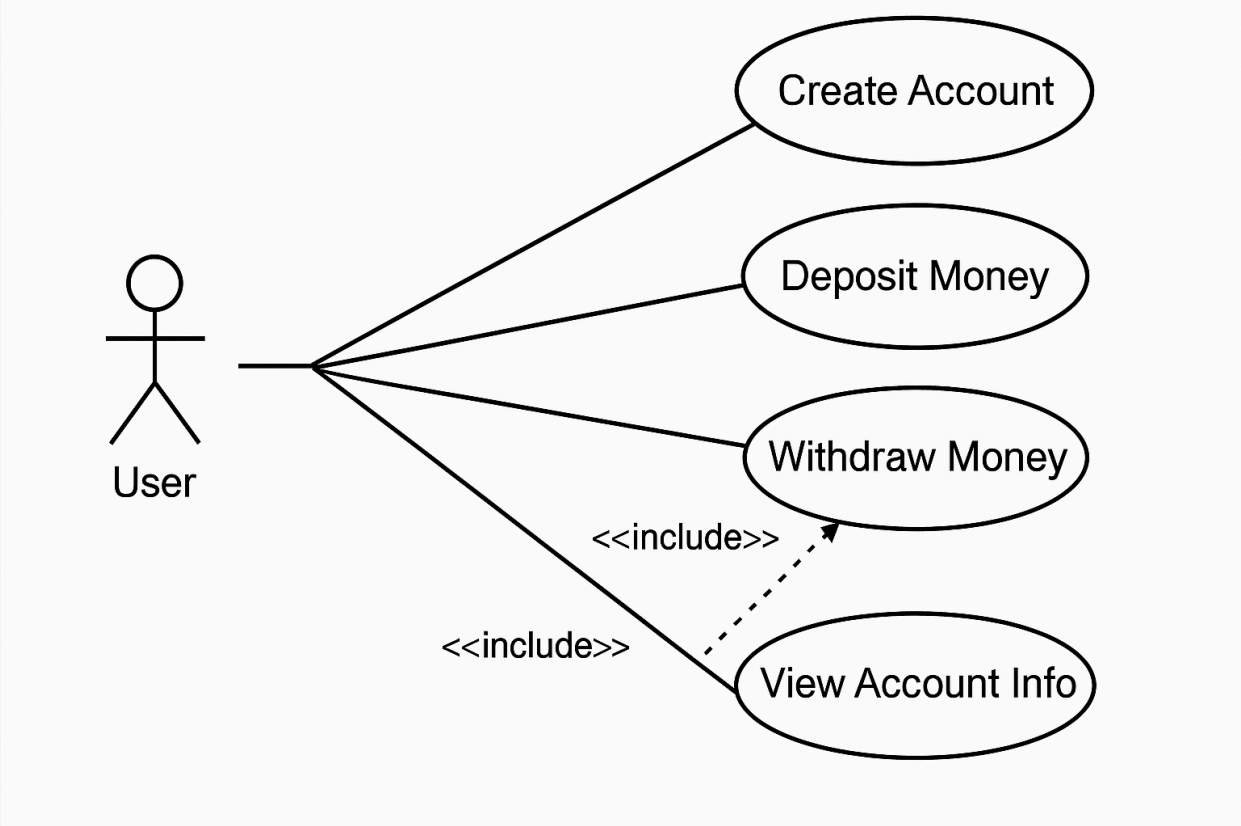
.

4.Relations(**include and exclude**) if needed.

Transfer Money ⬅️ *includes* → **Withdraw Money**

Transfer Money ⬅️ *includes* → **Deposit Money**

5.Perform steps 1-4 and convert it to **useCase diagram.**



Sequence Diagram

1.What are the **objects** involved in the banking system above?

The primary objects (instances of classes) involved in the system are:

- **Test** (initiator, representing the `main` method or user)

- **Account (acc1)** – sender account
- **Account (acc2)** – receiver account
- **Transaction** – handles the transfer between accounts

2. Observe and **find the sequence** of the code for sequence Diagram.

```
Account acc1 = new Account(...);            // Create sender account
Account acc2 = new Account(...);            // Create receiver account
acc1.displayAccountInfo();                  // Show sender info
acc2.displayAccountInfo();                  // Show receiver info
Transaction transaction = new Transaction();
transaction.transfer(acc1, acc2, 300.0);    // Transfer amount

Inside transfer():
    acc1.withdraw(300);                     // Withdraw from acc1
    acc2.deposit(300);                      // Deposit to acc2

acc1.displayAccountInfo();                  // Show sender after transfer
acc2.displayAccountInfo();                  // Show receiver after transfer
```

3. Are there any **synchronous and asynchronous** messages?

- All method calls in Java are **synchronous** by default (caller waits for completion).

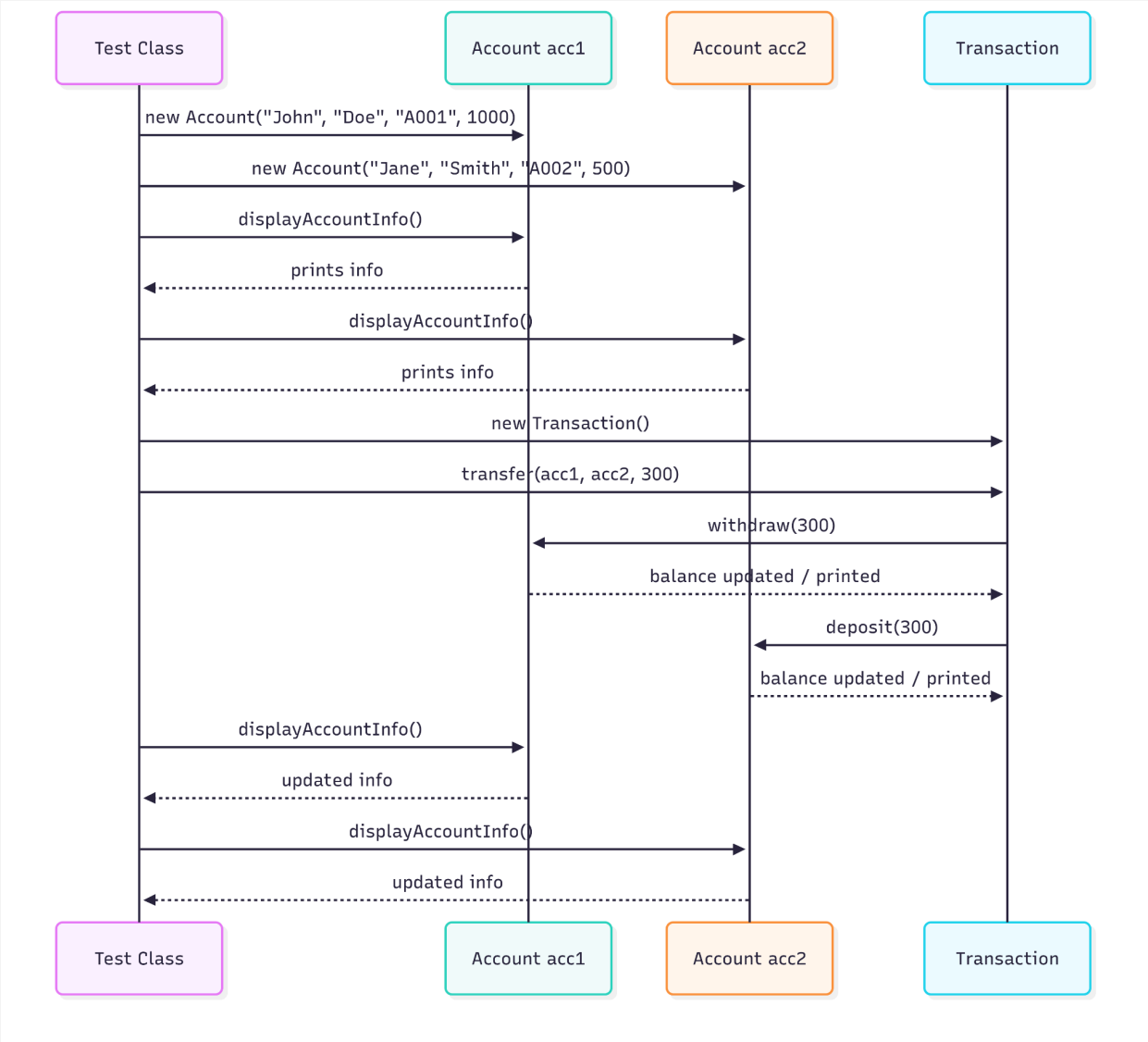
- No asynchronous messages are present in the given code.

4. What are the possible responses?

Each method may return a response like:

- Confirmation of method execution (e.g., "Withdrawn: 300", "Deposited: 300")
- Print statements (from `displayAccountInfo`)
- Console outputs
- No return values, just printed side effects

5.Perform steps mentioned above and convert it to **sequence Diagram?**


Test Class
Account acc1
Account acc2
Transaction
new Account("John", "Doe", "A001", 1000)
new Account("Jane", "Smith", "A002", 500)
displayAccountInfo()
prints info
displayAccountInfo()
prints info
new Transaction()
transfer(acc1, acc2, 300)
withdraw(300)
balance updated / printed
deposit(300)
balance updated / printed
displayAccountInfo()
updated info
displayAccountInfo()
updated info
Test Class
Account acc1
Account acc2
Transaction